

Contents

Corpus Construction.....	2
1. Language detection	2
1. English corpus creation.....	2
2. Text cleaning and tokenization	3
3. Review Length Boxplot	3
4. Keyword and length analysis	4
Sub-Study 1: Context, Collocation, and Topic Analysis	6
1. local_context_extraction.py	6
2. collocation_analysis.py	6
3. bertopic_analysis.py	7
2. Sub-Study 2: Sentiment Analysis.....	8
3. Sub-Study 3: GLiNER author mention extraction.....	9

Corpus Construction

1. Language detection

This script performs automatic language detection on Goodreads reviews of the six Kafka works "Amerika", "The Castle", "A Hunger Artist", "The Metamorphosis", "In the Penal Colony", and "The Trial". It prepares the corpus for the later filtering to English-language reviews.

The review files are read directly from **KafkaCSV.zip**. Language detection is applied to the "text" column using the "langdetect" library.

For each review, three new values are determined:

1. "language" -> detected language
2. "language_confidence" -> confidence score of the prediction
3. "language_note" -> detection status

Predictions with a confidence score of at least 0.80 are marked as "ok", while lower-confidence predictions are marked as "uncertain". Empty reviews, texts without alphabetic characters are marked as "unknown", if they exist.

The annotated datasets are saved in the "output" folder with the suffix **_annotated_confidence.csv**. The script also prints language counts for each Kafka work and an overall language distribution across all six datasets.

1. English corpus creation

This script creates the final English-language Goodreads corpus from the six previously annotated Kafka review datasets.

For each dataset, it standardizes the language labels, records the source file and book title, and retains only reviews classified as English with a language-detection confidence score of at least 0.80. Reviews classified as non-English, unknown, or English with confidence below 0.80 are excluded.

The filtered reviews from all six Kafka works are then combined into a single corpus and saved as **kafka_english_corpus.csv** in the output folder.

The script also reports the total number of retained reviews and the number of reviews for each Kafka work.

2. Text cleaning and tokenization

This script uses **kafka_english_corpus.csv** as its input file. It prepares the English Goodreads corpus for subsequent textual analysis by cleaning the review text and creating tokenized versions of each review.

The script reads **kafka_english_corpus.csv** created from **create_english_corpus.py**. If a "review_id" column is not already present, one is created so that each review can be uniquely identified. The review text is then cleaned by decoding HTML entities, removing "
" elements and other HTML tags, removing URLs, and normalizing repeated whitespace.

After cleaning, each review is converted to lowercase and tokenized using a regular-expression-based tokenizer. The total number of tokens in each review is also calculated.

Subsequently, three additional columns are created: "clean_text", which contains the cleaned review text, "tokens", containing the tokenized version of each review, and "token_count", which contains the number of tokens in each review.

The resulting corpus is saved as **kafka_english_corpus_tokenized.csv** in the output folder and is used as the cleaned and tokenized corpus for the subsequent analyses.

3. Review Length Boxplot

This script uses **kafka_english_corpus_tokenized.csv** as its input file. It outputs a boxplot regarding the distribution of review lengths in the tokenized English Kafka corpus. Review length is measured using the "token_count" value calculated for each review.

After reading `kafka_english_corpus_tokenized.csv`, the script prepares the token-count data, and groups reviews according to their Kafka work. The six works are ordered according to their median review length and are compared with the review-length distribution of the entire corpus.

A horizontal boxplot is created showing the interquartile range, median, mean, and outliers for each Kafka work and for the complete corpus. This makes it possible to compare differences in review length across the six works and to identify reviews with unusual lengths.

The resulting figure is saved as `boxplot_review_length_entire_and_books.png` in the output folder.

4. Keyword and length analysis

This script uses `kafka_english_corpus_tokenized.csv` as its input file. It analyses reviews containing Kafkaesque and related keyword variants in the cleaned and tokenized English Goodreads corpus. Reviews are identified through a case-insensitive regular-expression to search for Kafkaesque forms defined in the keyword pattern.

The script additionally calculates descriptive corpus statistics, including review counts, total token counts, unique type counts, and mean review length, for the complete corpus and each of the six Kafka works. It also compares keyword frequency across the six Kafka works and analyses the review-length distribution of Kafkaesque-containing reviews.

The reviews are divided into very short reviews (≤ 10 tokens), a main comparable corpus, and very long reviews above the $Q3 + 1.5 \times IQR$ boundary. The resulting datasets, descriptive and frequency statistics, and a review-length boxplot are saved for subsequent analysis.

The script produces these files located in the output folder:

`corpus_descriptive_statistics.csv`

`kafkaesque_keyword_reviews_all_books.csv`

kafkaesque_keyword_frequency_by_book.csv

kafkaesque_main_comparable_corpus.csv

boxplot_kafkaesque_review_lengths.png

Sub-Study 1: Context, Collocation, and Topic Analysis

These three scripts form the main pipeline for Sub-Study 1 and should be run in the order described below.

1. local_context_extraction.py

Input: kafkaesque_main_comparable_corpus.csv

Output: output/kafkaesque_local_contexts_all_six_books.csv

This script extracts the textual bits surrounding occurrences of "Kafkaesque" and its related forms from the main comparable Goodreads corpus, including one sentence before and one sentence after the keyword-containing sentence. The corpus needed to run this code, namely `kafkaesque_main_comparable_corpus.csv`, was created during the preprocessing step.

The script stores these contexts along with metadata such as the review ID, the book title, and the specific keyword variant used. All extracted local contexts are saved as a CSV file, which will serve as the main foundation for the subsequent analyses.

2. collocation_analysis.py

Input: output/kafkaesque_local_contexts_all_six_books.csv

Outputs: output/all_six_books_collocations_pmi.csv

and

output/all_six_books_collocations_pmi_top40.png

This script analyses the Kafkaesque local contexts by identifying meaningful word pairings using Pointwise Mutual Information (PMI). After loading the previously extracted local-context dataset, the script first splits the local contexts into individual sentences and removes repeated sentences within the same review, which prevents sentences shared by overlapping local contexts from being counted multiple times.

The remaining sentences are tokenized into lowercase word tokens and constructs bigrams from adjacent words. It counts both individual word frequencies and bigram frequencies. It also filters out any bigram that appears fewer than three times to avoid noise from extremely rare combinations.

For each remaining bigram, the script calculates PMI, a measure of how strongly two words are associated compared to what would be expected if they occurred independently. It also stores up to three example contexts for each bigram which allows interpretation of how these word pairs appear in real Kafkaesque descriptions. The resulting dataset is sorted by PMI and frequency, saved as a CSV file, and visualized in a bar chart showing the forty highest-PMI bigrams.

3. bertopic_analysis.py

Input: output/kafkaesque_local_contexts_all_six_books.csv

Output: output/bertopic_barchart.html

This script applies BERTopic to the extracted local contexts in order to identify topics regarding how Kafkaesque is described across the six Goodreads corpora. Each context is embedded using all-MiniLM-L6-v2 sentence-transformer model and then reduced with UMAP, using a fixed random seed of 42 for better reproducibility. Topic representations are generated from unigrams and bigrams that appear in at least two contexts, after removing standard English stop words and a set of work-specific terms such as “Kafka,” “Kafkaesque,” and other generic vocabulary. The resulting topics are visualized as an HTML bar chart that depicts the most representative words associated with each thematic cluster.

Words removed:

"kafkaesque",

"kafka",

"kafkaesk",

"kafkaian",
"kafkaesqued",
"kafkaesquely",
"kafkaesques",
"kafkaesqueity",
"esque",
"br",
"read",
"reading",
"book",
"story",
"like",
"just",
"im"

2. Sub-Study 2: Sentiment Analysis

Model: OpenAI GPT-5.6 Sol

Input: 1,638 Goodreads reviews containing Kafkaesque across six Kafka works.
(kafkaesque_main_comparable_corpus.csv)

Prompt used:

“analyse the given excel file of reviews containing the keyword "kafkaesque" from 6 Kafka books. Give me a description as to how the keyword "kafkaesque" is used and categorize the usage in each review according to its sentiment 1+0 and also take into account the 8 emotions by plutchik.”

Note: I initially provided a previously generated corpus to the model on Thu, Aug 20 at 2:44 PM, which the model processed. However, as this wasn't the corpus that was used for the overall analysis, it was discarded.

For this reason, only the output that was generated for the message at 3:32 PM, on 28/09/2026, reading "using the exact same prompt, do the same for this file." should be considered.

The entire chat history can be found here: <https://chatgpt.com/share/6aba752d-c514-83eb-acd5-ba11e9eab45f>

3. Sub-Study 3: GLiNER author mention extraction

Model: GLiNER Small v2.1 (`urchade/gliner\_small-v2.1`)

Input: English Goodreads reviews of The Metamorphosis from the csv file **kafka_english_corpus_tokenized.csv** which is found within the "input" folder located in sub-study 3

This script identifies and counts "author" mentions in Goodreads reviews of The Metamorphosis using the zero-shot Named Entity Recognition model GLiNER Small v2.1. The corpus is first restricted to reviews of The Metamorphosis, empty and duplicate review texts are removed, and the remaining reviews are split into chunks of up to 250 words for better processing due to the long length of some reviews.

GLiNER is applied using the zero-shot label "author" and a confidence threshold of 0.50. The label instructs the model to identify textual entities that it considers represents authors based on their surrounding context. Only detections with a confidence score of at least 0.50 are retained.

Each detected entity also includes a short slice of surrounding text of up to 120 characters, which allows for the identified entities to be inspected closely.

Since GLiNER tended to identify different forms of the same name as separate entities, frequently occurring name variants were manually grouped under a single name. For

example, "Albert" and "Camus" were grouped under "Albert Camus". Other entities that weren't given a grouping name were still retained.

Outputs:

1. metamorphosis_gliner_entities_raw.csv -> contains every individual entity detection that met the confidence threshold, including its review ID, book title, chunk number, extracted and normal name form, entity label, confidence score, and surrounding context. It was mainly used for the manual inspection of false positives and other ambiguous detections.

2. metamorphosis_canonical_entity_counts.csv -> contains the final entity counts after grouping together the different variations of the same entity and other detected entities, together with their total number of mentions, total number of distinct reviews in which they occurred, the average confidence score, and one example context. This file constituted the main aggregated output for sub study 3.